

Analytical evaluation methods

Most evaluation methods we will discuss in this part are *empirical*; that is, their primary sources of data are measurements and observations of real people. Although empirical methods offer the gold standard for evaluation, they are labor-intensive, costly, and at times ethically problematic. Evaluators therefore started to wonder if it would be possible to evaluate a design pre-empirically, that is, without involving users.

Analytic evaluation methods in human–computer interaction (HCI) are a class of evaluation methods that do not require the collection of data on (real) users. Their purpose is to assess the usability of a design and expose probable errors based on good questions, rules of thumb, or models of performance prediction with a user interface (UI). Generally, analytical methods consist of (1) a process for performing the evaluation and (2) a set of resources to be used in the process. The resources help the evaluator predict what problems a user might encounter when using an interactive system, which parts of a UI to focus on, or what the users’ performance with the system might be. Importantly, some of these resources serve as a yardstick against which to compare a UI or a design.

The main idea of an analytic method is simple to demonstrate. Consider the task of sending a message using your messaging app. First, take your mobile phone and open the messaging application. Choose a recipient and the message you would like to send. Then, execute this task. As you do this, write down on paper all the steps you needed to take: clicking, touching, scrolling, searching, deciding, and so on. Now, repeat the sequence, but this time *purposefully introduce one error* somewhere in the sequence. For example, you could press the wrong button when you are supposed to send the message. What happened? Did the interface try to prevent you from making an error, for example, by disabling that button? Did it inform you that there is an error? Did you easily recognize that you had made a mistake? If so, did the design help you recover from that error? These considerations exemplify the *heuristic evaluation guideline* called “Help users recognize, diagnose, and recover from errors”; see Section 41.1 for a comprehensive set of heuristics. Heuristic evaluation is one of the lightest-weight analytic methods, meaning that it is inexpensive and can be applied at all times throughout development.

Heuristics are *rules of thumb* that are often expressed as guidelines. They are imperatives that instruct how to design and how not to design, for example, *strive for consistency* [771]. HCI has a long history with guidelines, which have been developed for almost every interface technology, from computer terminals in the 1970s to AI systems in the 2020s. All major organizations have their design guidelines and associated design systems.

But where do guidelines come from? Guidelines, as all analytical methods discussed in this chapter, are inspired by either the experience of professionals working in the field or theories of cognition. Most design guidelines are based on expert experience. They summarize what works and what does not. Johnson [390] is a recent example of attempts to rethink guidelines from the cognition point of view. Johnson argued that proper rationale should be given for the guidelines.

Table 41.1 Practical design guidelines rooted in human cognition. Adapted from Johnson [390].

Guideline	Cognitive factor	Example
Design for a biased perception	Perception is biased by experience and goals	Placing the “OK” button in an unfamiliar place may make users not see it
Design for figure/ground perception	Perception evolved to see structure in environment	Structure information into wholes using visual cues such as proximity, closure, and common area
Design for easy scanning of text	A visual hierarchy (e.g., headings, bulleted lists, tables) helps form structure	Instead of writing out a paragraph of text, show it as a heading and a bulleted list of steps or points
Design for limited memory	Human working memory is limited and easily distracted	Avoid modal designs where the effects of input depend on the state of the interface, as the mode can be forgotten by the user
Design for goal-driven attention	Attention is biased toward goal-relevant items in an interface	Present task-relevant information in a way and position that align with the user’s ongoing task
Design for recognition, avoid insisting on recall	Recognizing stimuli is faster and less error-prone than actively recalling facts	Show options from which users can choose (e.g., icons, images) rather than requiring them to recall names (e.g., login names, passwords)
Design for goal-driven attention	Attention is biased toward goal-relevant items in an interface	Present task-relevant information in a way and position that align with the user’s ongoing task
Design for the right level of skill	Frequency, regularity, and the type of practice affect the level of skill that can be expected from the user	Use consistent terminology and labels throughout the user interface
Design for slips	A wrong routine can be incorrectly executed if the visual environment bears similarity with the right one	If a rare interaction has risks, make it visually dissimilar from routine ones

Many of them are directly rooted in an understanding of human cognition. Table 41.1 lists some guidelines based on Johnson’s book *Designing with the Mind in Mind*.

Analytical evaluation methods are commonly used for three purposes [31]:

1. In design, they are used to identify potential usability problems so that they can be rectified in design before deployment; this is a form of formative evaluation. For example, Langevin et al. [456] developed a heuristic evaluation method for conversational agents. The method can be used to identify potential usability problems of a prototype design. The heuristic set expands that of Nielsen and Molich [593], which has been popular in the evaluation of graphical user interfaces (GUIs); see Section 41.1.
2. In evaluation, they are used to assess usability against a baseline design or assess how ready a design is for deployment; these may be formative or summative evaluations. For example, a cognitive walkthrough can be used to assess whether users face problems in learning to use

a new system. Information systems for nurses, for example, often need to work intuitively. In an empirical study [238], five expert assessors evaluated a nursing information system. They identified 24 unique usability problems. The authors argued that by fixing some of the most critical issues, the learning time and cognitive load experienced by nurses could be reduced.

3. In accident investigation, they are used to identify factors that can increase the likelihood of accidents. Human error analysis, for example, is used in accident analysis in aviation [889].

Analytic methods are appealing because of their cost-efficiency: The savings can be remarkable compared to an empirical study. Unlike other forms of evaluation, such as think-aloud studies (Chapter 42), analytic evaluations are relatively quick to perform. The cost of an analytical evaluation primarily consists of the evaluators' salaries since not anyone can perform such evaluations. For high-quality evaluations, evaluators need to be trained in the methods. The higher the expertise, the better the results. Analytic methods can also be used on all types of representations of designs, from a system in use to an early mock-up of a design idea. Finally, analytic evaluation methods can help identify solutions to a problem. If a heuristic such as "Help users recognize, diagnose, and recover from errors" is breached, it may trigger ideas on how to fix the situation.

However, analytical evaluation methods are not a replacement for empirical studies. Compared to empirical studies, analytic methods tend to have high rates of false positives and false negatives. Cockton and Woolrych [160] found that 65% of the predictions from a heuristic evaluation were incorrect; Hvannberg et al. [368] found that 62% of the problems could not be detected in a usability test.

A common cause for the low hit rate is that their successful application heavily depends on the skill of the evaluator. If two evaluators were to perform the same analysis for the same case, their results might be different. In other words, *inter-evaluator agreement* tends to be low, leaving analytic evaluation methods vulnerable to the evaluator effect (Chapter 42). The limited scope of analytic methods is another threat to validity. Analytic methods pertain to a few predefined aspects of usability. Analytic evaluation methods do not reliably capture phenomena within their scope, and they never capture those outside their scope. For these reasons, analytical methods are not a replacement for empirical evaluations. They are best seen as complementary. When applied correctly, analytic methods decrease the cost of iterative design and increase its chances of success.

In the rest of this chapter, we look at five different approaches to analytic evaluation:

1. Heuristic evaluations, which capture theories and practitioners' experiences of what causes problems in interaction
2. HEI method, which is used to identify the potential for human error
3. Cognitive walkthrough, which builds on a theory of human cognition to propose how to evaluate the learnability of a design
4. Keystroke-level modeling (KLM), which is a simple mathematical model of experienced users' performance that predicts users' task completion time
5. Automated usability evaluation, where a yardstick for UIs is applied automatically to interactive systems.

Paper Example 41.0.1: Analytic evaluation for inclusiveness

Analytic methods are cost-efficient methods often considered to be limited to usability. However, plenty of methods have been developed for other factors that are important in HCI, including inclusiveness. How could one evaluate if a design is inclusive?

GenderMag, which is short for Gender Inclusiveness Magnifier [111], is an evaluation method created to investigate the inclusiveness of an interactive system. The idea is to combine personas (Chapter 15) with walkthroughs, which we discuss later in this chapter.

The method includes three customizable personas: Abi, Patricia/Patrick, and Tim. While Abi and Tim represent deliberately gendered personalities, Patricia/Patrick is decidedly neutral in those aspects and stronger in other facets (e.g., learning style). All three personas have five facets describing them in more detail: motivation, computing self-efficacy, risk attitude, information processing style, and learning. Self-efficacy here refers to beliefs about one's ability to complete computing tasks successfully. The evaluator starts by filling in these characteristics in light of user research data. They also walk through likely actions and relate them to the personas. In all steps, the evaluator can identify problems with the interactive systems.

An evaluative study of GenderMag reported a high positive rate in predicting a software's inclusiveness.

InclusiveMag is a generalization of GenderMag that considers not only gender but also eight diversity dimensions [536].

41.1 Heuristic evaluation

The term *heuristic* refers to a loosely defined rule, such as a rule of thumb. For example, if you love cooking, you may know a heuristic for checking whether an egg is fresh. If you place an egg in water and it floats instead of sinking to the bottom, it is likely to be old. In computer science, heuristics are rules for solving computational problems. For example, a useful heuristic in computer science is the following. A list of numbers can be sorted in increasing order by following this rule: "Pick any item: If it is smaller than the one before it, swap the two." If you continue applying this rule, eventually the whole list will be sorted. In HCI, heuristics refer to the best practices identified by practitioners working in the field. They are typically expressed as "dos and don'ts." Applied to a design, they can highlight potential problems in usability.

A *usability evaluation heuristic* is a rule for evaluating a UI. Such heuristics are used to detect probable *usability problems*. In a heuristic analysis of an interface, an *evaluator* is provided with a *set of heuristics* and the interface. A task is used to detect *breaches* of the heuristics; in this case, the heuristics serve as the yardstick for the evaluation. In case of breaches, the inference is that users will experience usability problems in using the system or that the usability of the system will be negatively affected.

Numerous heuristics have been presented for different platforms and uses, from GUIs for games to web pages [710]. The most popular heuristic evaluation method is attributed to work by Molich and Nielsen in the 1990s [554, 593]. They identified that a small set of guidelines covered much ground. The Molich and Nielsen heuristics are as follows.

Visibility of system status: The current state of the system should be visible to the user. A simple example is a progress bar that indicates the progress of a long-term operation (e.g., downloading a large file).

Match between system and the real world: The UI should follow the language and any relevant conventions users are already aware of. If a user has to look up a term to understand it, then usability is reduced.

User control and freedom: Users should be encouraged to explore different ways of achieving their goals in the UI. To allow this, it is important that users can reverse their actions. The ubiquitous *Undo* and *Redo* functions are examples of interface features introduced to support this heuristic.

Consistency and standards: First, UIs should follow standard platform, system, and industry conventions. This is sometimes known as maintaining external consistency. Second, similar interface features should be consistently labeled and visualized throughout the application or system. This is known as internal consistency.

Error prevention: The UI should be designed to prevent errors, for example, by displaying a warning and requiring user confirmation before a non-reversible action is triggered, such as deleting a file.

Recognition rather than recall: It is more difficult for users to recall from memory how to trigger an action than it is to recognize a mechanism for triggering the action shown on the display. Therefore, any information required to trigger common actions, such as labels, buttons, and menu items, should be either immediately visible or easily retrievable.

Flexibility and efficiency of use: Since users inevitably vary in their proficiency with a UI, it is often effective to provide interface features that can be tailored to different users. A simple example is providing keyboard shortcuts for menu items and toolbar buttons that offer an expert user immediate access to these functions, while a novice user can still use direct manipulation to easily locate them (albeit at a slower pace). Another strategy is to allow users to customize the UI or have the UI adapt to the user.

Aesthetic and minimalist design: Ensure that the UI focuses on content and information essential for allowing users to achieve their primary goals. Avoid providing information that is rarely relevant and avoid introducing UI elements that may distract users.

Help users recognize, diagnose, and recover from errors Provide error messages that are understandable by users and offer a clear solution path to rectify the problem.

Help and documentation: If documentation is required, ensure that it is focused on aiding users in their tasks and is easy to search. If possible, present documentation within the context it is required, such as a step requiring the user to make a decision. Any help is best provided by listing the concrete steps necessary for the user to carry out the task.

41.1.1 How to do a heuristic evaluation?

How to do a heuristic evaluation is best learned through an exercise. Take your laptop and open the settings panel of your operating system. Decide on a setting you want to change, such as the way your mouse cursor behaves (its transfer function), and the particular value you want to set it to. Now, go to the starting view, execute the required (i.e., correct) actions to change the setting, and write down all your steps on paper. Then, go through each step and compare it against each heuristic of Molich and Nielsen given previously. What to do with the result? First, count the number of heuristic violations. Is this an acceptable number? Note that counting violations does not tell us much about their real-world relevance. Second, think about the users of the system: What is important to them, and which violations would be harmful to them? Next, classify the violations into three classes of severity: (1) minor, (2) critical, and (3) catastrophic. If you have a single critical violation, you can consider the design to have failed.

Slightly more formally, the resources and methods that evaluators use in a heuristic evaluation are (1) the choice of heuristics and associated training materials, (2) a way of going through parts of the interactive system that is being evaluated, (3) a process for evaluation, and (4) analysis and reporting of the problems identified.

Choice of heuristics

The most common heuristics are the Molich and Nielsen heuristics described in the beginning of this section, which are deemed a safe option. Typically, each of these heuristics is associated with a more extensive explanation or teaching material. For example, the visibility of the system status concerns whether the user knows what an interactive system is currently doing and what it is possible to do with it. For instance, if the CAPS lock key is active, its status can be indicated by a light on it or a “CAPS LOCK ON” text field on a display. A full tutorial for the heuristics is available in Nielsen [591].

Heuristics that are *customized* for a particular use situation or user group can have better hit rates—but how to construct an effective heuristic set? Some heuristic sets are based on an authoritative expert’s view: “This works but this doesn’t.” This is fair—provided the heuristics are empirically validated and their scope is well known, it does not matter where they come from. However, to develop a set of heuristics that better captures consensus in a specific application area, several experts must be involved. This can be done pre hoc with several experts jointly developing heuristics or post hoc by clustering heuristics that experts have already developed. Specialized examples include heuristics for evaluating displays that are distributed in the environment [516], mobile computing [67], and the playability of games [189]. A longer worked example of heuristics developed for interactive AI is given in Paper Example 41.1.1.

Choice of system parts for evaluation

The original instructions for heuristic evaluation were unclear as to which parts of a system to evaluate and how to go through them [161]. As a result, several different approaches have emerged. The evaluator may follow a system scanning approach, that is, the evaluator scans the system, looking for a breach of heuristics. This approach helps cover most of the features of a system. Alternatively, the evaluator may use a set of tasks that are representative of what users may like to do with the interactive system. This helps to keep the evaluation focused on what users likely want to do and uncover dependencies and inconsistencies in a large task. While the choice is up to the evaluator, the task approach is often preferred because it helps keep false positives somewhat better in check.

Which heuristic to consider and when? Some guidelines recommend focusing on one heuristic at a time, as it is relatively easy to scan a system or go through a task while keeping just one heuristic in mind. An alternative is to consider all heuristics at each step of a task or for each feature. Again, the choice is up to the evaluator. The former option has a higher risk of false positives, for example, an evaluator might report a breach of a heuristic because they are focusing on that heuristic; the latter option is harder for the evaluator, who may forget some heuristics.

Process of evaluation

A heuristic evaluation is typically a stand-alone process. It is preferable to use several evaluators who first work individually and then combine and discuss their results together. One reason for this is attention to reliability: As individual evaluators may not spot the same problems, it is useful to combine the work of several evaluators. This is a general approach to improving reliability and is the reason why, for example, a questionnaire may include multiple questions about the same construct (Chapter 13). Another reason is that if the evaluators worked together from the start, they might bias each other to see the same usability problems.

Paper Example 41.1.1: Heuristic evaluation of interactive AI

Heuristic sets can be developed for a given class of technologies. As discussed in the book, this can be done through expert panels and empirical studies. A timely example is heuristics for evaluating systems that use interactive AI. Amershi et al. [19] proposed a list of 18 heuristics:

1. Make clear what the AI system can do
2. Make clear how well it can do what it does
3. Time services based on context
4. Show contextually relevant information
5. Match relevant social norms
6. Mitigate social biases
7. Support efficient invocation
8. Support efficient dismissal
9. Support efficient correction
10. Scope services when in doubt
11. Make clear why the system did what it did
12. Remember recent interactions
13. Learn from user behavior
14. Update and adapt cautiously
15. Encourage granular feedback
16. Convey the consequences of user actions
17. Provide global controls
18. Notify users about changes.

An example is given in the Paper Example figure.

Heuristic evaluation of an autocomplete feature

phone phone number phone number lookup phone phone cases phone repair phone lookup phone number for verizon phonetic alphabet phone repair near me phone number for comcast	Violations of Amershi et al.'s AI evaluation heuristics Show contextually relevant information • Suggestions can be irrelevant to task Mitigate social biases • Suggestions can be inappropriate Support efficient dismissal • Must go to settings to turn off Support efficient correction • No way to correct or edit suggestions Make clear why system did what it did • Unclear where the suggestions come from
---	---

Heuristic evaluation of an autocomplete feature in text entry. Adapted from Amershi et al. [19].

Analysis and reporting of usability problems

Based on the heuristics and the choice of the system, an evaluator makes a list of notes on the problems. The actual work of using the heuristics to identify problems is often called analysis.

Analysis is often underappreciated in usability evaluation [251]. However, because heuristics are just resources that an evaluator judiciously applies, considering and analyzing the potential problems are important steps. Does it seem plausible that a particular potential problem is *actually* a problem? What is the cause of the problem? What could be done, if anything, to resolve the problem? A couple of resources and concepts can help in the analysis of problems.

- **Frequency.** It is useful for an evaluator to assess how often a problem would occur for prospective users. One commonly used scale separates problems into three categories: rare, occasional, and frequent.
- **Severity rating.** This expresses, according to the evaluator, how serious a problem is for users. One commonly used rating scale is as follows. (1) Minor: The user is temporarily delayed. (2) Serious: The user is delayed significantly but can eventually complete the task. (3) Catastrophic: Prevents the user from completing a task.
- **Persistence.** This assessment concerns whether the problem is a one-off occurrence that users will learn to work around or something that will bother them repeatedly.
- **Cause.** What is the cause or reason for the problem? Even if an issue is pointed out by a heuristic, it may not be clear what in the interactive system is behind the problem. This is an even bigger problem in empirical evaluation (Chapter 42), where mapping users' expressed frustration to issues in the system may be very difficult.
- **Redesign suggestion.** This asks the evaluator to consider how the interactive system might be redesigned to avoid the problem. Redesigns can help evaluators avoid reporting problems that have no solution. This task also impacts the clarity and usefulness of the problem to other people [354].

The reporting of problems has been shown to be an important step in the analysis of problems. The above items, for instance, can be simultaneously addressed using a structured reporting format, such as that shown in Table 41.2.

41.1.2 Reliability of heuristic evaluation

Heuristic evaluation has a role in the quick assessment of designs; however, it is not a panacea or a replacement for empirical studies. The known benefits and drawbacks of heuristic evaluation include:

- **High efficiency:** Some usability problems can be spotted with little effort and experience.
- **Limited scope:** Heuristics are limited to aspects of usability that can be attributed to visible parts of the UI.
- **High false positive and false negative rates:** Many important problems are not found, and some identified problems are not problems.
- **No guarantees:** The results cannot be trusted to be comprehensive, reliable (due to the large variability), or generalizable (except for obvious usability problems).
- **High variability:** Evaluators, even experienced ones, have drastically varying hit rates.

Table 41.2 Example of a usability problem reporting format.

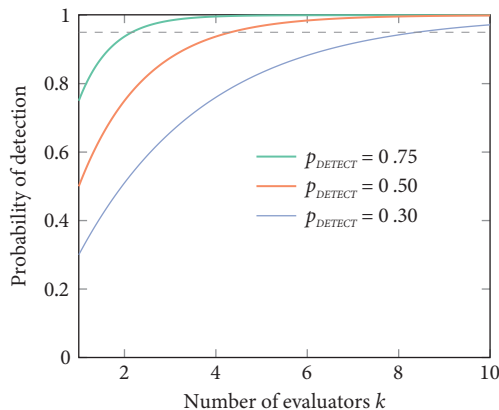
Description	Difficulty for user	Cause	Frequency	Severity	Solution
"Open report" is hidden	The user needs to search the entire page to find the link	The link is located in a spot that the user might not notice	Frequent: The user will experience this difficulty every time they log in	Serious: The user may struggle to find the link for a while	Move the link to the upper-left corner

One core reason for these issues, particularly high variability, is that evaluators are imperfect as “signal detectors.” Every time an evaluator inspects some facet of a design, with some probability, some problem is found. However, this can be a false positive or a false negative. In other words, the evaluator may have found an error where there isn’t one, or they may have missed an actual error. The more complex the system and the more problems, even obvious ones, the more the evaluators will miss. A statistical analysis illuminates this phenomenon.

From a statistics perspective, an attempt to use a heuristic to detect a violation can be thought of as an *experiment*. It is a trial with two possible outcomes: success or failure. You can think of it as analogous to the tossing of a *biased coin*: A usability problem is either detected (success) or not (failure). In statistics, experiments with two outcomes are called Bernoulli trials. We use Bernoulli trials here to answer a central question in heuristic evaluation: What is the proportion of problems that we expect k evaluators to be able to find?

Number of evaluators

We start with the case of a single usability problem and ask how the number of evaluators k affects the probability of its detection. Let us assume that our k evaluators are imperfect. They can spot a real usability problem with probability $0 \leq p_{\text{DETECT}} \leq 1$. We further assume that *if* an evaluator detects a problem, the problem exists. In other words, for simplicity, there are no false positives in this model.



Now, let the random variable $\mathbf{X}$ denote the number of evaluators needed such that *at least one* detects the usability problem. This follows the Bernoulli distribution $\mathbf{X} \sim B(p)$. In a Bernoulli process, a Bernoulli trial is repeated k times. In this case, we are interested in the occurrence of *any* true positive within that sequence, which is given as $p(\mathbf{X} > 0) = 1.0 - p_{\text{DETECT}}^k$.

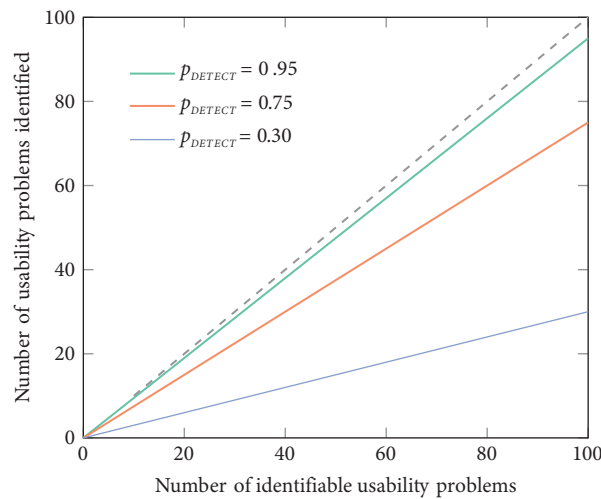
The plot shows that we only need two expert evaluators (red line) to detect a usability problem with high probability (≥ 0.95), increasing to four or more when the evaluators are intermediately

skilled (blue). Note that for a novice, such as a computer science student, the detection probability ranges between 7% and 75% [593] and is often around 30%. The black line shows that seven or more novices would be required for reliable detection. In summary, the skill of the evaluator matters.

Coverage of problems

We can now look at the case where we have several usability problems. Let us assume that a system has n usability problems. We ask a group of equally skilled evaluators to evaluate it and assume their individual detection probability is p_{DETECT} .

Let the random variable X denote the number of usability problems observed by the evaluators together. The number of problems found in n trials with detection probability p_{DETECT} follows a *binomial distribution*: $X \sim B(n, p_{\text{DETECT}})$. Then, the number of usability problems found is np_{DETECT} .



Putting the first and second results together, we learn the following:

1. High coverage of real problems is hard to achieve without experts (red). Only experts provide a high probability of detection.
2. Evaluations by a single evaluator are inherently unreliable. Even an expert evaluator will miss an unacceptably large proportion of “obvious” usability problems (blue).
3. Even a novice evaluator will find *some* usability problems.

These analyses show that heuristic evaluation is not a silver bullet, nor does it replace empirical evaluations. While well-trained experts can detect a nontrivial proportion of true problems, the analyses suggest exercising caution when applying heuristic methods and emphasize the need for rigorous evaluation of the results.

41.2 Identification of human error potential

Human error identification (HEI) is a class of analytic evaluation methods for identifying the possibilities of human error in interaction (see Figure 41.1). Although HEI techniques were originally developed for safety-critical applications, the main idea generalizes to most state-based UIs. It can be used to identify interactions where users might be confused or make a mistake.

One key difference from heuristic evaluation is that HEI is preceded by task analysis (Chapter 15). In practice, this means that the states and goals of an interface are first identified. A user is then “simulated” going through the states toward a goal to investigate what kinds of errors might happen.

Consider a vending machine and the task of buying a soda can. This task is actually quite complex. It consists of several steps, such as finding the desired soda on the list, pressing the button, finding the wallet, finding the credit card, inserting it, entering the PIN, waiting for the soda, and picking it up. The basic idea of HEI is to look at the *transitions* between steps. What can go wrong when moving from one step to another? For example, why would a user not find the credit card or insert it into the wrong place? HEI can reveal possibilities for taking wrong steps.

A central concept in HEI is *state*. At any given time, the interface can be in one state where it is ready to receive input from the user. Some actions are available; others are not. The system communicates its state and possible next actions on its display, for example, via text or graphics.

To utilize HEI, we first enumerate the states of the UI. We then form a matrix showing possible transitions between states. We label each cell to show whether a transition is available: (1) legal, (−1) illegal (erroneous; should not be done by user), and (0) not available.

This state matrix is used as the basis for simulating users. Take a user persona and a particular task for that persona. Then, starting from the state in which the user would encounter the device, enumerate the reasons why the user might take a *wrong action* in that state. HEI suggests three main types of errors, but in principle any cause of error is valid:

1. User confuses the correct action with something else
2. User confuses the state of the machine
3. User selects erroneously.

For each illegal (wrong) action, assess the consequences of that error. What would happen, and how high would the cost be?

Can HEI techniques actually predict errors? To answer this question, Baber and Stanton observed public vending machine use for 24 hours, totaling 300 observations of transactions [31]. They categorized and tabulated the found usability problems; the most common problems concerned money insertion (27%), zone or destination selection (22%), and ticket type or travel card selection (21%). This formed their ground-truth dataset. Independently of these observations, they carried out HEI. Their task analysis diagram identified 11 subtasks, sometimes reaching a depth of three levels of task, reflecting the nontriviality of the machine. Error analysis was performed by taking a state and enumerating all available actions. The transitions caused by these actions were then mapped to other states, forming a matrix. The matrix marks legal (1), illegal (−1), and not available (0) transitions.

Their analysis exposed the following problems with a vending machine, which may be familiar to many of us:

1. Not understanding what to do
2. Selecting wrong ticket
3. Selecting wrong station
4. Selecting wrong zone
5. Problems in inserting money
6. Confusing mode

7. Pressing wrong buttons
8. Using a machine that is closed
9. Confusing return from cancel
10. Use a machine that is waiting.

Comparing these findings with the ground-truth dataset, the authors concluded that one evaluator using the method could identify more than 80% of the real usability problems. This analysis took about 3.5 hours per evaluator, representing significant time savings compared to conducting real-world observations.

41.3 Cognitive walkthrough

Cognitive walkthrough is an analytical evaluation method based on the mental simulation of the way users think (see Figure 41.2). It is an instance of a broader class of *walkthrough methods* used across technical disciplines, for example, architectural walkthroughs in architecture and code walkthroughs in software engineering.

In a cognitive walkthrough, an artifact is inspected systematically and evaluated against some criteria. In this sense, it is similar to HEI. What makes cognitive walkthrough special is that the evaluation criteria are related to thinking and cognition. The method relies on the evaluator *simulating* the interaction *in their mind* to determine whether a user *might* succeed or fail. The user is simulated as guessing and exploring how to use an interface. However, walkthroughs are not just any form of imagery; they follow systematic procedures and conceptual apparatuses.

The goal of a cognitive walkthrough is to expose possible problems impairing the *ease of use and learnability* of a system. The method is recommended for understanding how novice users may learn to use a system. Its scope is different from that of other methods discussed in this chapter, which focus on user performance, errors, and usability problems.

A cognitive walkthrough is straightforward but substantially more laborious than a heuristic evaluation. The inputs to the method are (1) the UI, (2) a task scenario that defines what the users are supposed to accomplish, (3) assumptions about the users and the contexts of use, and (4) a sequence of actions for completing the tasks. Task analysis is needed to prepare point (4). In most cases, the analysis of subtask *sequences* is sufficient.

There is good evidence that cognitive walkthroughs can predict a significant part of learnability-related problems. In the comparative study by Lewis et al. [474], cognitive walkthrough detected 50% of problems exposed in an empirical user study.

41.3.1 How to do a cognitive walkthrough?

In a walkthrough session, tasks are demonstrated to a team in a step-by-step manner, attempting to explain, plausibly, how the user might tackle four issues related to use:

1. Will the user try to achieve the right effect?
2. Will the user notice the availability of the correct action?
3. Will the user associate the correct action with the intended effect?
4. If the correct action is performed, will the user be aware that the task is progressing as intended?

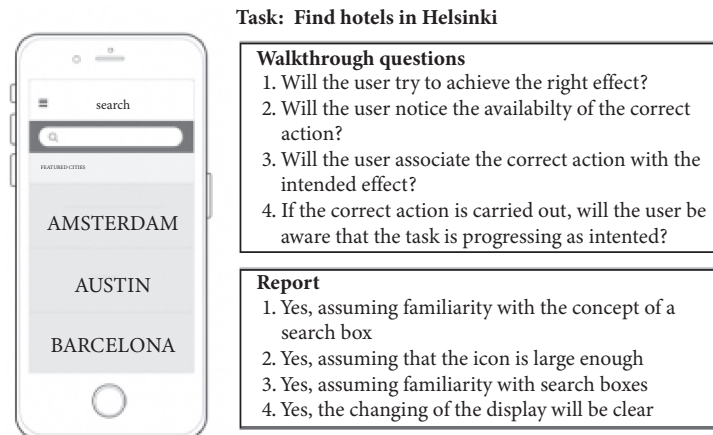


Figure 41.2 Cognitive walkthrough: Simulating a user solving a task step-by-step.

If a plausible explanation cannot be given, this is recorded as a critical issue. These critical issues are reviewed together with the design team to identify design gaps and set goals for the next design iteration.

41.3.2 A theory of how people learn via exploration

Cognitive walkthrough is rooted in a theory of how people learn interfaces. Informally, it could be called a theory of how people *guess* what to do next. The theory of *cognitive exploration* was proposed by Polson and Lewis [671]. The crux of the theory is this: To complete a task, the user must set a goal relevant to the task and achieve its subgoals by taking the necessary actions. For each subgoal, the user must *select* the right action, *execute* it correctly, and *confirm* that the interaction is progressing as desired.

To solve this, two gulfs must be crossed: the *gulf of evaluation*, or determining the right action for the goal, and the *gulf of execution*, or successfully executing the action and confirming that it was a success (Chapter 18).

According to Polson and Lewis [671], people first establish the *goal structure*, which associates subgoals to the top-level goal. For example, when cooking a pasta dish, the subgoals are related to the acquisition of ingredients, the preparation of the cooking space, familiarization with the recipe, and so on. However, the goal structure may be incomplete. Missing subgoals must be figured out on the go, which puts further emphasis on how well the UI can guide the user.

A key part of the theory concerns the representation of a subgoal. A subgoal is a representation associated with (1) other subgoals, (2) actions, and (3) perceptual cues. Consider the subgoal of turning on a stove. This subgoal is associated with the top-level goal of making tea as well as the previous and following subgoals (e.g., putting the kettle on the stove). It is associated with actions that are needed to accomplish the goal, such as turning a knob. It is also associated with perceptions or cues, such as those of the knob, the stove, and the kettle. According to the theory, these cues activate the representation of the associated goals and actions: Seeing the knob is sufficient to activate the subgoal of turning on the stove. Once the subgoal has been

achieved, the representation is deactivated. Thus, a representation can be *activated* and *deactivated* (inhibited) by its associated concept. When interaction flows well, percepts activate the right subgoals, which activate the right actions. When taking the right action, the state changes such that the next subgoal is activated, and so forth. When there are no actions that are highly activated, the user must explore and try something. This inevitably leads to errors. However, the erroneous paths will then be deactivated, and a better goal structure will be learned over time.

One of the benefits of the theory is that it helps to address some issues in design prior to evaluation. Lewis et al. [474] proposed four design tactics for improving guessability:

1. Make the repertory of available actions salient
2. Provide an obvious way to undo actions
3. Offer few alternatives
4. Require as few choices as possible.

A cognitive walkthrough is more time-consuming than a heuristic evaluation, and it typically requires a trained evaluator. Deployment in software teams has been reported to face problems due to time pressure [787]. Similar to heuristic evaluation, the high rate of false negatives in cognitive walkthroughs is a recognized problem. While cognitive walkthrough complements heuristic evaluation, neither method is comprehensive. Cognitive walkthroughs are best used in early, noncritical stages and should be carried out by experienced evaluators.

Several adaptations of the method have been created for various purposes. An overview of cognitive walkthrough variants and extensions is provided by Mahatody et al. [510]. A leaner version has only two questions [787]:

1. Will the users know what to do at this step?
2. If they take the right action, will they know that they did the right thing and are making progress toward their goal?

41.4 Keystroke-level modeling

The KLM is a simple mathematical model used to assess the performance of tasks. In particular, it can predict the *task completion time* of experienced users. KLM was introduced by Card, Moran, and Newell in their 1983 book *Psychology of Human-Computer Interaction* as a simplified version of operators, methods, and selection rules (GOMS) [129], a more comprehensive model based on a cognitive architecture (Chapter 5).

KLM deals with tasks that have sequential substructures, such as form-filling, text editing, data entry, or setting manipulation. KLM is limited to sequential tasks with clear task boundaries, no parallelisms (e.g., multitasking), and few dependencies between subtasks. By assuming that performance in each subtask is independent of performance in the others, the task completion time can be modeled as the sum of the time spent on the subtasks.

KLM is keystroke-level in the sense that it assumes nothing about higher-order control nor does it assume subtasks influence each other. In other words, behavior is deterministic, going from one subtask to another, and the task completion time is the sum of the time spent on “atomic” responses and actions. There is no learning or memory assumed in the user. Every atomic response takes

place regardless of what happened in the past. Although KLM was derived from GOMS, it is not a cognitive model but a performance evaluation model. It does not say anything deep about how the mind works in interaction. Despite being simple, the model works well for UIs that are sequential in nature.

41.4.1 How to do a KLM analysis?

KLM starts with the analysis of the user's task to find the most likely way (or ways) of accomplishing it. This task performance is broken down according to three categories of *operations*: physical, mental, and system. Physical operations refer to actual actions and movements of the user, say, pressing a key or clicking a mouse. Mental operations refer to events such as recalling a command name or verifying that an answer is correct. One system operation is the system response time—the time spent waiting. These operations are counted, and the time spent on each is estimated using guidelines and look-up tables. The task completion time (T) is their linear sum:

$$\begin{aligned}
 T &= t_K \text{ } \textit{keystroking} \\
 &+ t_P \text{ } \textit{pointing} \\
 &+ t_H \text{ } \textit{homing} \\
 &+ t_D \text{ } \textit{drawing} \\
 &+ t_M \text{ } \textit{mental operation} \\
 &+ t_R \text{ } \textit{system response.}
 \end{aligned}
 \tag{41.1}$$

These values have been empirically estimated [127]. *Keystroking* is 0.12–1.2 s, 0.28 s for most users, and is affected by the level of expertise. An expert typist has an average keystroke time of 0.12 s, whereas someone with less exposure to keyboards has 1.2 s. *Clicking* a mouse button takes 0.20 s. *Pointing* averages at 1.1 s, although with varying movement distances, should be determined using Fitts' law. *Homing* is the act of moving the hands between the mouse and the keyboard. It takes about 0.4 s. *Mental operations* are in the range of 0.6–1.35 s; the recommended value is 1.2 s. Mental operations are required when initiating a task, making a strategy decision, recalling something from memory (e.g., password), finding something on the screen, thinking about what to do, or verifying the correctness of inputs. The *system response time* varies and must be measured.

KLM includes eight steps [129]:

1. Choose one or many representative task scenarios.
2. Specify the design to the point that keystroke-level actions can be listed for the scenarios.
3. For each scenario, identify the most likely ways users will accomplish the tasks.
4. List the keystroke-level actions and the physical operators involved in doing the task.
5. If necessary, include wait operators for the time when the user is waiting for the system to respond.
6. Insert mental operators for when the user has to stop and think.
7. Look up the standard execution time of each operator.
8. Sum up the execution times of the operators. This is the estimated task completion time.

41.4.2 Worked example

Let us consider a simple example with physical, mental, and system operations. An expert user is editing a document and wants to replace all instances of the word “might” with the word “will.” Table 41.3 shows a breakdown of the actions, which can be summarized as follows:

$$\begin{aligned}
 T &= t_K + t_P + t_H + t_M + t_R \\
 &= (0.60 \text{ s} + 0.48 \text{ s} + 3 \times 0.20 \text{ s}) + (3 \times 1.10 \text{ s}) + \\
 &\quad (5 \times 0.40 \text{ s}) + (3 \times 1.20 \text{ s}) + 0.50 \text{ s} \\
 &= 11.08 \text{ s.}
 \end{aligned}$$

The analysis reveals that the user spends almost one-quarter of the time just moving the hand between the mouse and the keyboard.

41.4.3 Limits of KLM

In empirical studies, KLM has proven to be effective at predicting the task completion time within a large but acceptable limit of tolerance. In one evaluation, KLM predictions were compared against the performance of 20 expert users [397]. Across six tasks used in the evaluation and UIs, KLM predicted users’ performance in four tasks. The average difference between predictions and users’ performance was 5.5 s (range: 0.8–13.35 s).

The time spent on system operations can be measured, and the time spent on physical operations can be estimated from observational data. However, how to measure mental operations? Best practices in KLM suggest that it is more important to get the type and number of operations

Table 41.3 Example KLM model of a replace task in text editing.

User action	KLM event	Time (seconds)
Reach for mouse	<i>H</i> : mouse	0.40
Search “Replace”	<i>M</i> : visual search	1.20
Move pointer to “Replace” button	<i>P</i> : menu item	1.10
Click on “Replace” command	<i>K</i> : click	0.20
Home on keyboard	<i>H</i> : keyboard	0.40
Type word to be replaced “might”	<i>K</i> : type	0.60
Reach for mouse	<i>H</i> : mouse	0.40
Point to correct field	<i>P</i> : field	1.10
Click on field	<i>K</i> : click	0.20
Home on keyboard	<i>H</i> : keyboard	0.40
Recall word	<i>M</i> : recall	1.20
Type new word “will”	<i>K</i> : type	0.48
Reach for mouse	<i>H</i> : mouse	0.40
Move pointer on “Replace All”	<i>P</i> : button	1.10
Click on field	<i>K</i> : click	0.20
Wait	<i>R</i> : system	0.50
Verify	<i>M</i> : verify	1.20
Total		11.08

right than to determine the order in which they occur. Moreover, very different interfaces should not be compared without first identifying suitable values for M . For instance, command-language interfaces and GUIs have different requirements for recall, so using the same constant for both will bias the estimates.

Several criticisms have been leveled at KLM, including the following [619]. First, KLM assumes expert performance—a user who is able to directly use the most efficient strategy and carries out tasks without error. In this respect, it complements cognitive walkthrough, which focuses on how people “guess” what to do. Second, KLM ignores flexibility in human activity. In Chapter 21, we learned about behavioral strategies and how they are adapted to the structure of a task. Full-fledged cognitive models are needed when behavioral adaptation is a defining part of users’ performance. Third, KLM ignores the variation in performance. It is known that there are dramatic inter- and intra-individual differences even in the lowest levels of actions, such as button-pressing. KLM collapses that variability into a single-point estimate. If the statistical distributions of KLM operators were known, they could be used to sample value combinations. There have been attempts to collect such distributions for different factors, such as age [279]. Fourth, KLM assumes no parallel activities in the mind, even though multitasking is a pervasive aspect of computer use.

41.5 Automated usability evaluation

In recent decades, much research has attempted to eliminate the need for an evaluator and automate usability evaluation [377]. The core idea of automated usability evaluation is that an evaluation tool encapsulates a yardstick for good interaction. The yardstick may be that an interactive system is accessible, that it is easy to perceive the information on the display, that links are not broken, and so on.

For example, Aalto Interface Metrics [628] takes a web page as input and computes its accessibility, aesthetics, and support for accurate color perception, among other aspects. The evaluator of a web page may use these descriptors to make inferences about how well the web page supports users. As another example, the Web Accessibility Initiative maintains a list of tools that help automatically check accessibility (see <https://www.w3.org/WAI/>). These tools typically take a web address as input and create a list of possible or likely issues with accessibility. However, their scope is limited compared to HEI, cognitive walkthrough, and KLM.

41.5.1 Interactive modeling workbenches

Computational cognitive models simulate human cognition through the stepwise execution of a program. Generally, computational models are superior to KLM and cognitive walkthroughs due to their ability to simulate cognitive states, strategies, and motor actions. They can provide insights beyond mere task completion times, shedding light on what constitutes successful and failed task performance. Generally, these tools are best suited for modeling skilled users.

However, one obstacle to their use has been that the task procedure must be provided by the researcher. At a minimum, this involves a demonstration of a sequence of actions. Interactive modeling workbenches allow us to specify a procedure by simply demonstrating it on a UI.

For example, *CogTool* is a modeling workbench that uses a version of GOMS for predicting how attention, motor control, and memory work during interaction [279, 388]. After uploading screenshots of the UI, the evaluator can demonstrate a task sequence, which is then simulated.

Comparisons against empirical results suggest a reasonable predictive power. In a study comparing CogTool's predictions against human data in 108 web-based tasks, CogTool's predictions accounted for 63%–82% of the participants' variance in success rate, number of clicks, and error rates [817].

Distract-R is a simulation workbench for multitasking in driving [725]. It uses another model called ACT-R (Adaptive Control of Thought–Rational) with a model of multitasking called threaded cognition. Evaluators can specify a secondary UI (e.g., media player UI), some limited aspects of the driving task (e.g., speed), and user details (e.g., age). *Distract-R* then computes task completion and lane deviation predictions and provides a video of the simulated driving.

41.6 Which analytical evaluation method to use?

Analytic methods excel at identifying usability problems; however, no serious project should rely *just* on them, especially for critical conclusions on safety, product launch, inclusivity, or accessibility. Analytic methods should be viewed as a powerful and flexible complement to empirical methods.

Which analytic method to use, and when? To understand the relative merits of each method, Blandford et al. [77] conducted one of the most comprehensive comparisons of analytic methods so far. They compared eight methods in the case of a human–robot interaction task. They proposed a few dimensions to consider when picking an analytical method:

Scope: What kinds of usability problems should be found?

Suitability: What type of interaction and user group is in question?

Reliability: What is the minimum reliability that should be achieved?

External validity: How important is the transfer of findings to real use?

Efficiency: How much information is gained per unit of resource use?

Persuasiveness: How to communicate the results in a way that convinces the audience?

Blandford and colleagues concluded that no single analytic method offered superior coverage of problems; rather, each method conquered its own niche in the space of usability problems. Their main conclusions are as follows. First, heuristic evaluation identified a broad range of issues but was most unreliable. The interpretation of the heuristic (e.g., what is “consistent”?) is left to the evaluator, producing high inter-rater variability. On the positive side, the openness of heuristics leaves room for a more qualitative consideration of the causes and consequences of issues, which can be useful for design. Second, cognitive walkthrough complemented other methods by exposing issues related to user misconceptions, consistent with cognitive exploration theory. Surprisingly, it encourages the identification of more issues than those strictly within its scope, making it similar to heuristic evaluation, which is relatively open-ended. Third, cognitive models, especially GOMS and KLM, best support the identification of system-related problems, such as the lack of undo functionality, redundant operators, or long action sequences, as well as some problems in synchronizing users' actions with those of the system. Model-based methods tend to focus on timing information, for example, how long actions or tasks take to complete. This is due to the focus on skilled users and the limited consideration of errors, learning, and behavioral variability.

Summary

- Analytic evaluation relies on expert evaluators systematically going through a design and procedurally checking system responses against some criteria.
- Analytic methods have low reliability when applied outside their scope or by inexperienced evaluators.
- There are analytic methods for covering different aspects of usability, from performance to errors and learnability. Walkthrough methods are being developed to assess the inclusiveness of a design against persona-based criteria.
- Analytic methods are best utilized as cost-efficient complements in early design stages; they are not a replacement for empirical evaluation methods.

Exercises

1. Heuristic evaluation. Pick a simple UI, such as a travel planner, the homepage of a volunteer organization, or a magazine app. Conduct a heuristic evaluation individually and report the identified problems following the format of Table 41.2. While doing the evaluation, note down difficulties, questions, and insecurities about the method.
Next, answer the following questions and, if possible, discuss your answers with peers who have also done an evaluation of the same interface.
 - What was hard in doing the evaluation and what was easy?
 - If other people have evaluated the same interface, do your evaluations agree? Do the three most critical problems overlap? Why or why not?
 - Given how you and your peers did the evaluation, what are the three main concerns about the validity, reliability, and impact of your evaluation?
 - An open question in research on analytic evaluation methods is whether evaluators see problems and then justify them with a heuristic or whether they use the heuristic actively to identify problems. Which description best fits your work and why?
2. Cognitive walkthrough. Do a cognitive walkthrough of the interface from the previous exercise. Then, answer the same questions as in the previous exercise.
3. KLM. Consider two designs for the same task, for example, two designs for logging into an information system. Conduct a KLM analysis of the two alternatives and compare them. What aspects that might affect task completion time are not covered by the KLM analysis?
4. Designing with analytical models. Take the UI you analyzed in the previous exercise and redesign it to improve the predicted task completion time.
5. Comparison. Consider the cognitive dimensions framework presented in Chapter 27. Is this an analytical evaluation method? Why or why not?
6. Designing with KLM. A smartphone app enables a user to achieve a task through two alternative methods. In the first method, the user must push a button on the screen, navigate to a menu with five choices, and choose the fifth choice. This brings up a Yes/No confirmation dialog box. The user must select Yes. In the second method, the user must hold a physical button down for one second until the system plays a brief audio beep and then speak a command. This results in the system playing a different audio beep to indicate it has commenced

speech recognition, and yet another different audio beep to indicate speech recognition has concluded. Assuming speech recognition has recognized the command, the system uses audio to ask the user for confirmation. Again, the user has to hold a physical button down for one second until the system plays a brief audio beep and then say “Yes.”

- (a) Carry out a KLM analysis to understand the time durations required to carry out the task using the two methods. Propose any operators necessary and estimate their time durations. Briefly motivate the operators, their estimated time durations, and the operator sequences involved.
- (b) Explain the limitations of the KLM analysis carried out in (a) with reference to the following design issues: (i) novice versus expert performance and (ii) the uncertainties inherent in interaction when attempting to carry out the task using the two methods.
- (c) An undesired event may occur when the user attempts to carry out the task. This can happen when, upon having to confirm the action, the user indicates “No.” Draw a fault tree (Chapter 37) to investigate the possible causes of such an event.